

INGEGNERIA DEL SOFTWARE · INGEGNERIA
INFORMATICA · UNISA

Casi d'Uso e UML

Appunti di ripasso e materiale d'esame — Lezione 4

Casi d'uso

Attori

UML

Extend / Include

Specializzazione

Strumenti

Fonte del corso

Slide "Lezione 4 — Casi d'Uso e UML", P. Ritrovato — N. Capuano, DIEM, Università di Salerno.

Testo di riferimento: M. Fowler, *UML Distilled*, 4^a ed., Pearson, Capitoli 1 e 9.

Rielaborato e ampliato per lo studio e la preparazione dell'esame.

Indice

1. Casi d'uso

Cos'è un caso d'uso · Attori · Esempio completo (AcquistaBiglietto) · Flussi alternativi · A cosa servono i casi d'uso

2. Introduzione a UML

Cos'è UML · Significato dell'acronimo · Perché usare UML · Forward/reverse engineering · UML come sketch, blueprint, linguaggio di programmazione

3. Diagrammi dei casi d'uso

Elementi base · Relazione «extend» · Relazione «include» · Relazione di specializzazione · Quando usare quale relazione · Esempio completo Hospital Reception · Errore comune · Critiche di Fowler

4. Strumenti e plugin

Strumenti di disegno generali · Diagram as Code (PlantUML, Mermaid) · Strumenti di modellazione UML

5. Riepilogo per l'esame

Mappa concettuale in breve · Domande di autoverifica con risposta

1 · Casi d'uso

1.1 Cos'è un caso d'uso

DEFINIZIONE I **Casi d'Uso** (*Use Case*) sono un modo di descrivere le interazioni tra utenti e sistema utilizzando un **testo strutturato** e un **modello grafico**.

La descrizione di un caso d'uso consiste in sei elementi fissi:

- Un **nome univoco**
- **Attori** partecipanti
- **Pre-condizioni**
- **Post-condizioni**
- **Flusso di eventi normale**
- Eventuali **flussi di eventi alternativi**

1.2 Attori

DEFINIZIONE Un **attore** è una tipologia di utente che interagisce con il sistema. Esempio: nel sistema informativo dell'università, sono attori gli studenti, gli addetti alla segreteria, i docenti.

- Gli attori **partecipano** ai casi d'uso: un attore può partecipare a più casi d'uso, e un caso d'uso può prevedere la partecipazione di più attori diversi

ATTENZIONE, PUNTO D'ESAME Un attore **non è necessariamente una persona**: se un sistema usa servizi offerti da altri sistemi, questi ultimi sono considerati a tutti gli effetti attori.

1.3 Esempio completo: AcquistaBiglietto

Nome: AcquistaBiglietto

Attore partecipante: Passeggero

Precondizioni:

- Passeggero in piedi davanti al distributore di biglietti
- Passeggero ha denaro sufficiente per acquistare il biglietto

Postcondizioni:

- Passeggero ha il biglietto

Flusso di eventi:

1. Passeggero seleziona il numero di zone da percorrere
2. Il distributore visualizza l'importo dovuto
3. Passeggero inserisce una somma di denaro, pari almeno all'importo dovuto
4. Il distributore restituisce il resto
5. Il distributore emette un biglietto

MANCA QUALCOSA? I CASI ECCEZIONALI! Il flusso descritto sopra copre solo il caso "tutto va bene". Un caso d'uso completo deve prevedere anche cosa succede quando qualcosa va storto: da qui i **flussi di eventi alternativi**, numerati in relazione al passo del flusso principale da cui si diramano.

Flussi di eventi alternativi (dallo stesso esempio)

Passo	Descrizione
3a	Passeggero annulla l'operazione
3a.1	L'esecuzione riprende dal passo 1
3b	Trascorrono 60 secondi senza che Passeggero inserisca alcuna moneta
3b.1	Il distributore restituisce l'eventuale denaro già inserito
3b.2	L'esecuzione riprende dal passo 2

COME LEGGERE LA NUMERAZIONE "3a" e "3b" indicano due possibili alternative che si diramano dal **passo 3** del flusso normale (l'inserimento del denaro): la prima per un annullamento volontario, la seconda per un timeout. I numeri successivi (3a.1, 3b.1, 3b.2) descrivono la sequenza propria di ciascuna alternativa, fino a un punto di rientro nel flusso principale o alla fine del caso d'uso.

1.4 A cosa servono i casi d'uso

- **Elicitare o descrivere i requisiti** — la caratterizzazione dei casi d'uso spesso porta alla scoperta di nuovi requisiti man mano che il sistema viene analizzato e il progetto prende forma
- **Comunicare con i clienti** — la semplicità della notazione rende i casi d'uso un buon modo per gli sviluppatori di comunicare con i clienti
- **Scegliere i casi di test** — la caratterizzazione di un caso d'uso può fornire indicazioni su come definire i casi di test che ne verificheranno l'implementazione

COLLEGAMENTO CON LA LEZIONE 3 I casi d'uso sono a tutti gli effetti una delle tecniche di **definizione dei requisiti** viste nella lezione precedente, complementare al linguaggio naturale strutturato: dove le tabelle IPO descrivono un singolo passo input→output, i casi d'uso descrivono l'intera interazione, comprese le sue varianti ed eccezioni.

2 · Introduzione a UML

2.1 Cos'è UML

DEFINIZIONE UML (*Unified Modeling Language*) è uno standard **OMG** (Object Management Group), basato sul lavoro di **Booch, Rumbaugh e Jacobson**. È un linguaggio di modellazione per **specificare, progettare e documentare** sistemi software.

- È un **linguaggio grafico**, che definisce un insieme di diagrammi e notazioni grafiche
- Particolarmente utile per la **progettazione orientata agli oggetti**
- **Indipendente** dal linguaggio di programmazione
- **Non è** una metodologia né un processo di sviluppo software

2.2 Il significato dell'acronimo

Parola	Significato
Linguaggio	Serve a esprimere concetti e idee in maniera precisa e non ambigua
Modellazione	Descrive un sistema software ad un alto livello di astrazione
Unificato	UML è diventato uno standard riconosciuto a livello globale

2.3 Perché usare UML

- Standard aperto, notazione grafica per specificare, visualizzare, costruire e documentare sistemi software — basato sull'esperienza e sulle esigenze della comunità di sviluppatori
- Migliora la **comprensione/comunicazione** del prodotto da parte di clienti e sviluppatori: può essere più chiaro del linguaggio naturale (impreciso) e del codice (troppo dettagliato)
- Può contribuire ad acquisire una **visione d'insieme** di un sistema

Testo di riferimento consigliato dal corso: M. Fowler, *UML Distilled* (4th ed.), Pearson — si concentra sui concetti e sui diagrammi più importanti di UML.

2.4 Come utilizzare UML

Gli sviluppatori utilizzano UML per comunicare alcuni aspetti di un sistema, secondo due direzioni possibili:

Approccio	Descrizione
Forward engineering	Si disegnano i diagrammi UML <i>prima</i> di scrivere il codice (ad esempio, in fase di progettazione)
Reverse engineering	Si disegnano diagrammi UML a partire da un codice <i>esistente</i> , per aiutarne la comprensione

Tre modi di usare UML, dal più libero al più rigido

Modalità	Obiettivo	Livello di dettaglio
UML come schizzo (sketch)	Comunicare alcuni aspetti, non la completezza; ci si concentra solo sulle questioni importanti	Incompletezza intenzionale : non si descrive tutto il codice né tutti i dettagli
UML come disegno tecnico (blueprint)	Costruire uno schema dettagliato che un programmatore possa tradurre direttamente in codice	Deve essere sufficientemente completo e chiarire tutte le decisioni progettuali
UML come linguaggio di programmazione	I diagrammi vengono tradotti automaticamente in codice eseguibile (parzialmente supportato da alcuni IDE con plugin specifici)	I diagrammi devono essere completi ed estremamente dettagliati, rappresentando anche le decisioni più banali

UNA DOMANDA SCOMODA, POSTA APPOSTA DAL CORSO Perché disegnare diagrammi piuttosto che scrivere direttamente il codice? Ha senso solo se la creazione dei diagrammi con il livello di dettaglio richiesto richiede **meno tempo** della scrittura in un linguaggio di programmazione convenzionale — **cosa che di solito non succede**. È per questo che nella pratica UML viene usato quasi sempre in modalità "schizzo": la modalità "linguaggio di programmazione" resta un caso di nicchia.

3 · Diagrammi dei casi d'uso

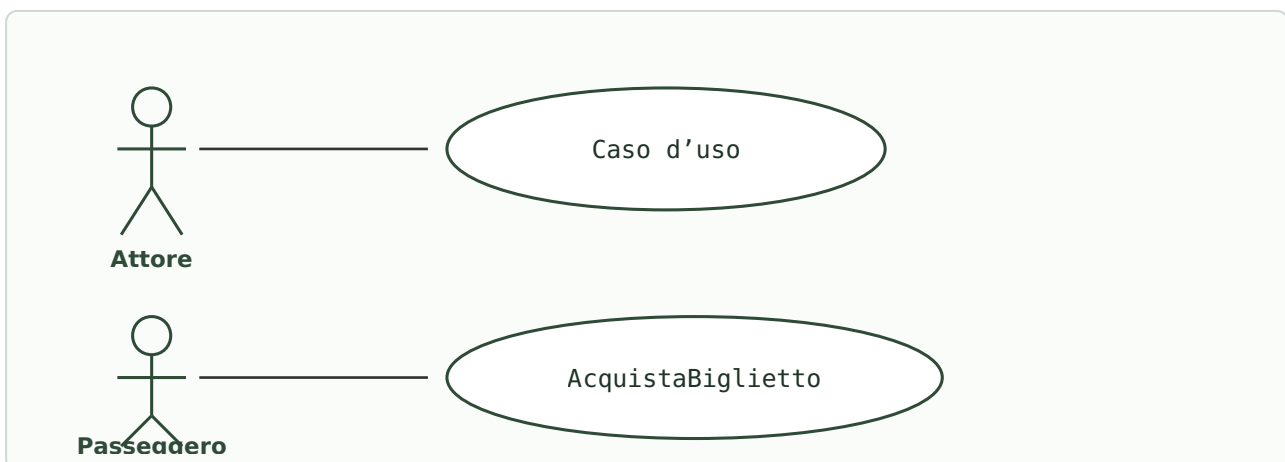
3.1 Cos'è un diagramma dei casi d'uso

DEFINIZIONE Un **diagramma dei casi d'uso** (*Use case diagram*) rappresenta una vista d'insieme dei casi d'uso di un sistema: evidenzia le **relazioni tra attori e casi d'uso** e le **relazioni tra casi d'uso collegati**.

ERRORE FREQUENTE Un diagramma dei casi d'uso **non sostituisce** le descrizioni dei casi d'uso viste nella Sezione 1: è utile solo se è accompagnato dalle descrizioni testuali vere e proprie, che restano la fonte primaria di informazione.

3.2 Elementi base di un diagramma

Gli elementi fondamentali sono due — l'**attore** (rappresentato come un omino stilizzato) e il **caso d'uso** (un'ellisse con il nome all'interno) — collegati da una linea di **associazione** che indica la partecipazione dell'attore al caso d'uso:

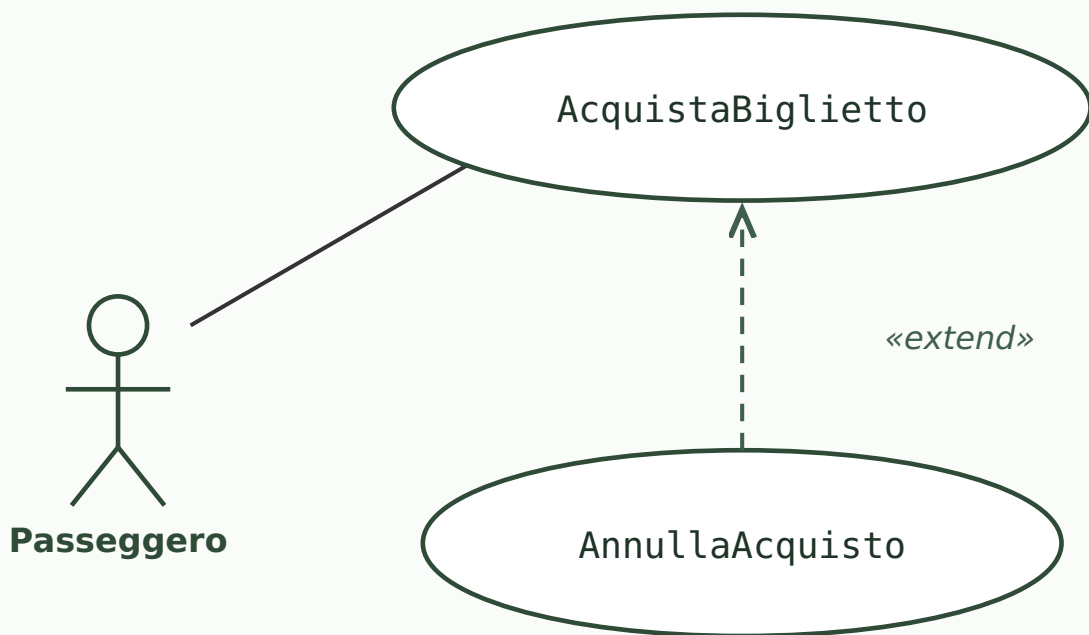


In alto: notazione generica. In basso: l'esempio "Passeggero — AcquistaBiglietto" già visto nella Sezione 1.

La linea che collega un attore a un caso d'uso indica semplicemente che **quell'attore partecipa a quel caso d'uso** — senza specificare altro sul tipo di interazione.

3.3 La relazione «extend»

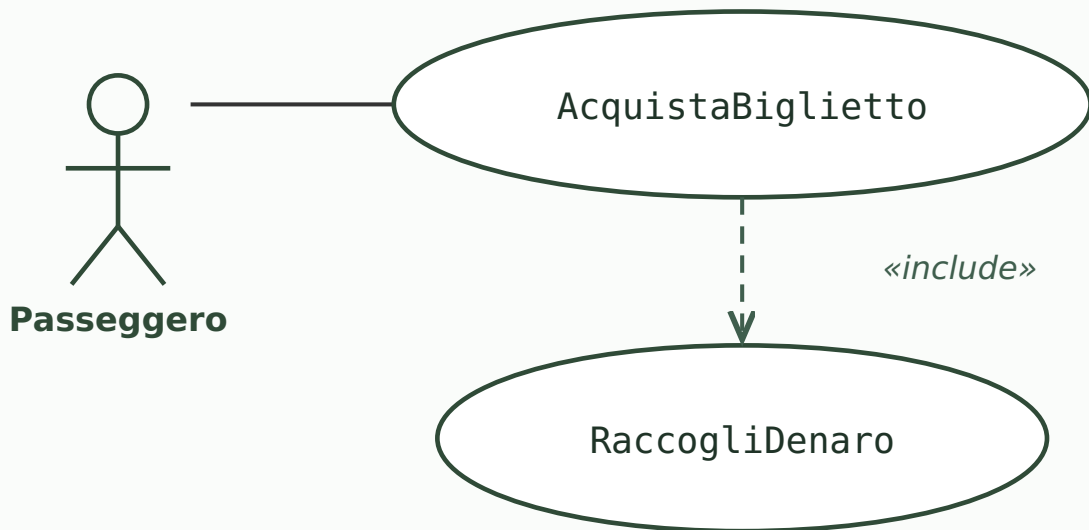
- Le relazioni «extend» rappresentano **casi eccezionali o raramente invocati**
- I flussi di eventi eccezionali sono **estratti** dal flusso dell'evento principale per motivi di chiarezza
- I casi d'uso che rappresentano flussi eccezionali possono estendere **più di un** caso d'uso
- La **direzione** di una relazione «extend» è verso il caso d'uso **esteso** (il caso base)



L'annullamento è un caso eccezionale: **AnnullaAcquisto** estende **AcquistaBiglietto** (freccia tratteggiata verso il caso esteso).

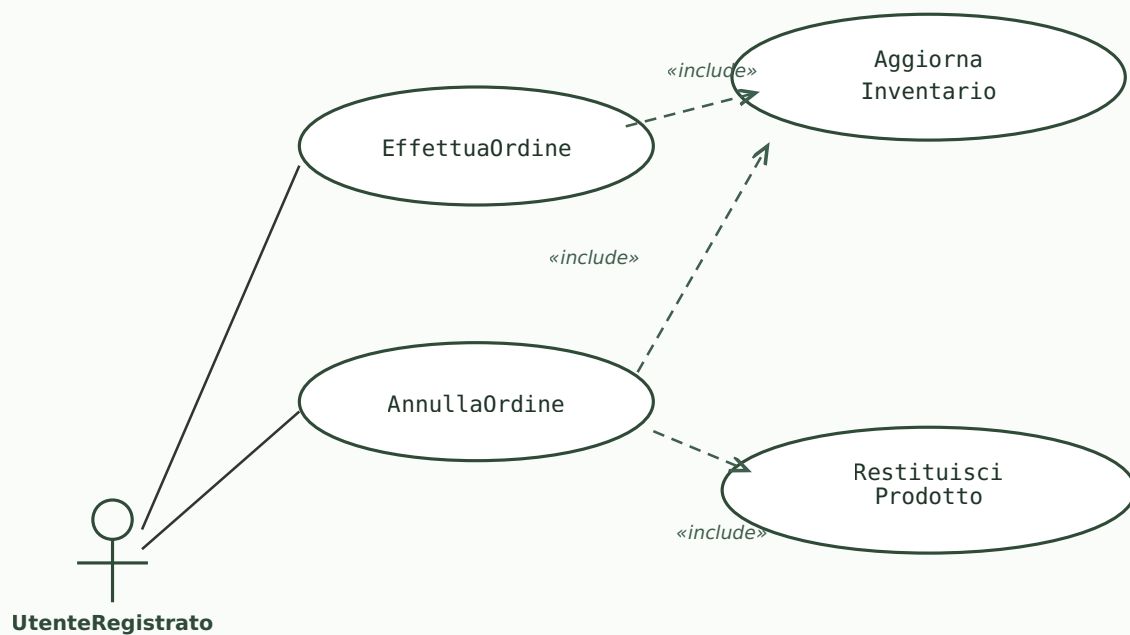
3.4 La relazione «include»

- La relazione «include» indica che un caso d'uso è un **passo** di un altro caso d'uso
- L'estrazione ha lo scopo di: **semplificare** la descrizione del caso d'uso principale, e **riutilizzare** il caso d'uso incluso
- La **direzione** di una relazione «include» è verso il caso d'uso **utilizzato** (incluso)



Raccogliere il denaro è un passo sempre necessario di AcquistaBiglietto: la freccia tratteggiata punta verso il caso **incluso** (RaccogliDenaro).

Un secondo esempio, con due casi d'uso che condividono un passo



Sia effettuare sia annullare un ordine richiedono di aggiornare l'inventario; annullare un ordine richiede anche di restituire il prodotto.

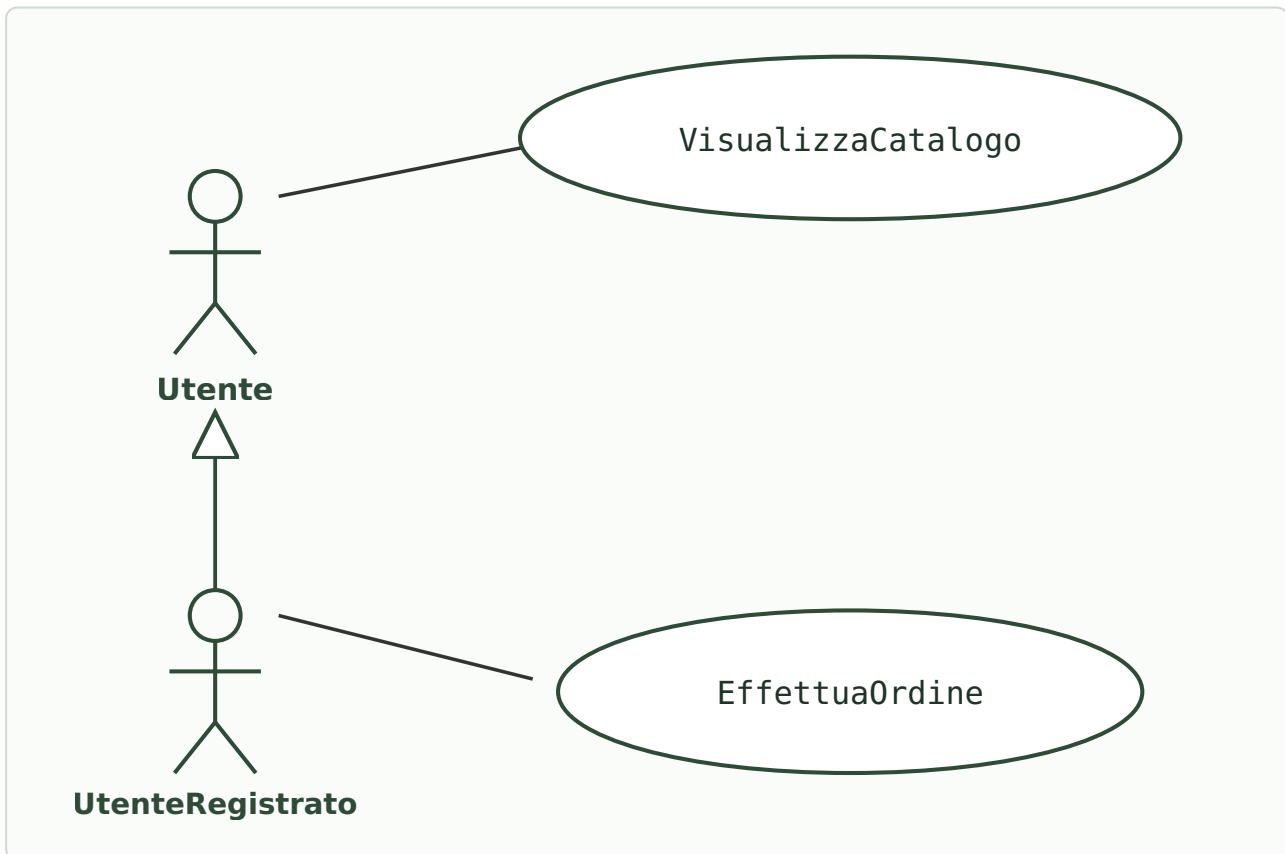
3.5 La relazione di specializzazione

- La relazione di **specializzazione** (detta anche, se "letta al contrario", di **generalizzazione**) indica che un elemento è un **caso particolare** di un altro elemento
- Nei diagrammi dei casi d'uso è usata principalmente per gli **attori**
- La **direzione** di una relazione di specializzazione è verso l'elemento **più generale**



Passeggero Abbonato è un caso particolare di Passeggero: la freccia (triangolo vuoto) punta verso l'elemento più generale.

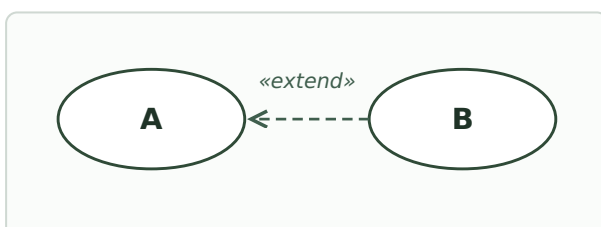
Un esempio con eredità delle associazioni



IL PUNTO PIÙ SOTTILE DI QUESTA SEZIONE Poiché **UtenteRegistrato** è un caso particolare di **Utente**, non c'è bisogno di indicare esplicitamente che **UtenteRegistrato** può partecipare a **VisualizzaCatalogo**: quell'associazione viene **ereditata automaticamente** dalla relazione di specializzazione, esattamente come l'ereditarietà tra classi in programmazione a oggetti.

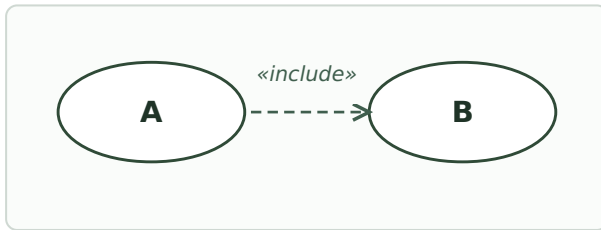
3.6 Quando usare le tre relazioni

Un errore comune è usare il tipo di relazione sbagliato tra due casi d'uso A e B. La differenza si gioca tutta sul significato semantico della freccia, non sulla forma del diagramma:



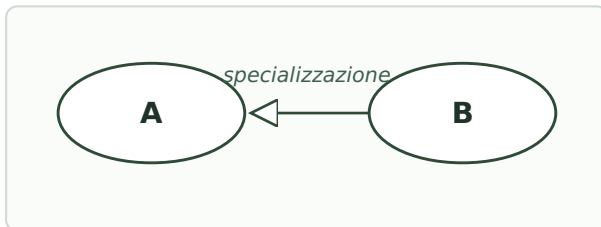
B estende A quando, al verificarsi di determinate condizioni (quindi **non sempre**), l'esecuzione di A comporta *anche* l'esecuzione di B.

Esempio: **AnnullamentoOperazione** estende **OperazioneBancomat**, perché il Cliente *può decidere* di annullare l'operazione (ma di solito non lo fa).



A include B quando l'esecuzione di A comporta **sempre** anche l'esecuzione di B.

Esempio: OperazioneBancomat include InserimentoPIN, perché il Cliente deve *sempre* inserire il suo PIN (anche se lo ha già fatto per un'altra operazione).



B specializza A quando B si può considerare un caso particolare di A; l'esecuzione di B **sostituisce**, in determinate situazioni, l'esecuzione di A.

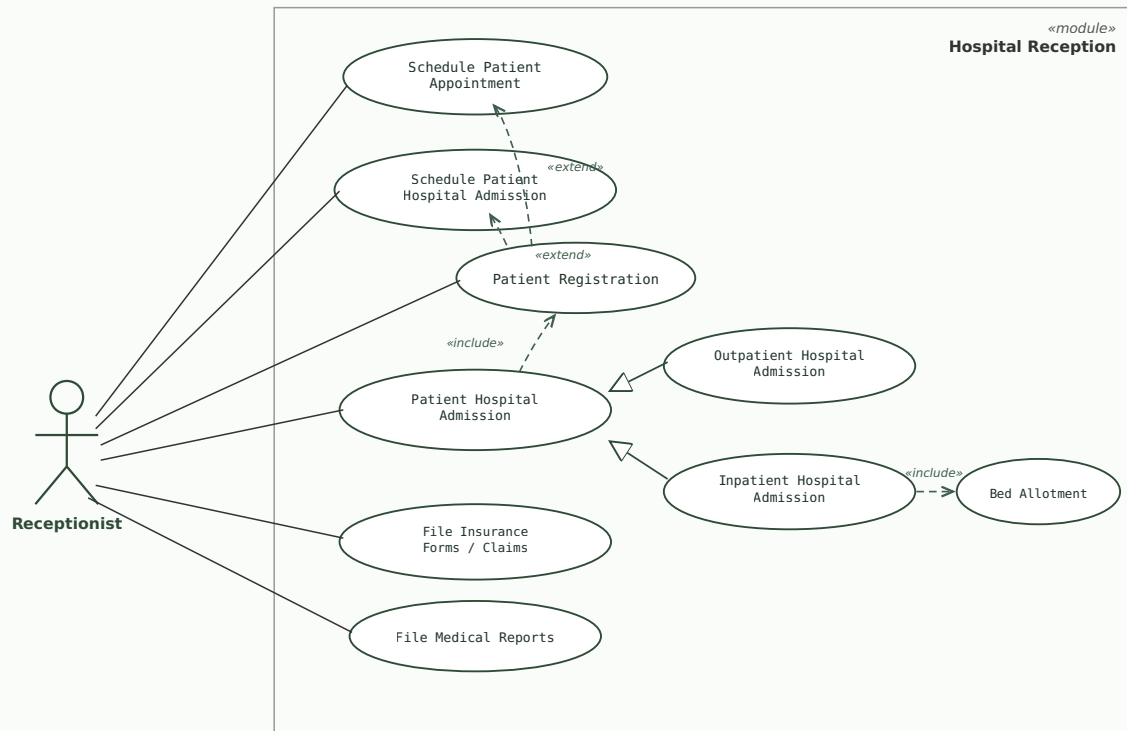
Esempio: PrelievoBancomat specializza OperazioneBancomat, perché è una delle operazioni possibili che il cliente può scegliere.

COME DISTINGUERLE IN UN COLPO D'OCCHIO «**extend**» = eccezione facoltativa (B a volte si aggiunge ad A). «**include**» = passo obbligatorio (A ha sempre bisogno di B).

Specializzazione = alternativa (B è un tipo particolare di A, e lo sostituisce). Nei tre casi la freccia punta sempre verso "il caso più generale/di base" (A), ma il tipo di linea e il significato cambiano completamente.

3.7 Diagramma dei casi d'uso: esempio completo (Hospital Reception)

Un esempio più realistico, che combina attore, associazioni e tutte e tre le relazioni viste finora all'interno di un unico **confine di sistema** (il rettangolo etichettato «module» Hospital Reception):



EXTEND Patient Registration estende **Schedule Patient Appointment** e **Schedule Patient Hospital Admission**, perché il Receptionist può eseguire la registrazione del paziente in questa fase, ma non è obbligato (lo farà eventualmente dopo).

SPECIALIZZAZIONE Outpatient Hospital Admission e Inpatient Hospital Admission specializzano **Patient Hospital Admission**: sono rispettivamente il caso speciale di ammissione in day-hospital e quello per i pazienti da ricoverare.

INCLUDE Patient Hospital Admission include **Patient Registration**, perché in questa fase la registrazione è obbligatoria.

INCLUDE Inpatient Hospital Admission include **Bed Allotment**, perché l'ammissione di un paziente ricoverato comporta sempre l'assegnazione di un posto letto.

3.8 Errore comune: attenzione al tipo di linea

Poiché UML è un linguaggio, ha delle regole precise di sintassi. Un errore comune è sbagliare il **tipo** o la **direzione** di una linea/freccia usata per indicare una relazione, cambiando così il significato del diagramma. Queste quattro linee hanno un significato **completamente diverso** tra loro:



Linea solida + triangolo vuoto — relazione di generalizzazione/
specializzazione



Linea solida semplice — associazione (attore ↔ caso d'uso)



Linea tratteggiata + freccia aperta, etichetta «include» — relazione
di inclusione



Linea tratteggiata + freccia aperta, etichetta «extend» — relazione di
estensione (identica graficamente all'inclusione: **l'etichetta è l'unica
differenza visiva!**)

3.9 Le critiche di Martin Fowler

"Più osservo i casi d'uso, meno utile mi appare il diagramma dei casi d'uso. Quando lavorate con i casi d'uso, è meglio concentrare le vostre energie sul testo, piuttosto che sul diagramma. Sebbene UML non riguardi il testo del caso d'uso, è proprio nel testo che risiedono le informazioni più preziose. Un rischio comune è complicare eccessivamente i casi d'uso, portando le persone a bloccarsi nel processo."

— Martin Fowler

3.10 Riepilogo della sezione

- I casi d'uso rappresentano il **comportamento esterno** di un sistema
- I diagrammi dei casi d'uso sono utili come **indice** dei casi d'uso
- Le **descrizioni** dei casi d'uso forniscono le informazioni più importanti, **non i diagrammi**

4 · Strumenti e plugin

4.1 Strumenti di disegno generali

Strumenti di uso generale che includono le forme UML tra le loro librerie grafiche:

- **Microsoft Visio** (per Windows)
- **Omnigraffle** (per Mac)
- **LibreOffice Draw** (open source, multiplatforma)
- **draw.io** (gratuito, online — drawio.com)
- e molti altri...

4.2 Diagram as Code

Strumenti per generare diagrammi UML a partire da **testo strutturato** invece che disegnandoli manualmente — entrambi open source e utilizzabili gratuitamente online:

Strumento	Caratteristiche
PlantUML (plantuml.com)	Aspetto più tradizionale, copertura di UML completa, plugin disponibili per molti IDE (tra cui NetBeans)
Mermaid (mermaid.live)	Stile moderno, conformità UML parziale, manca il plugin per NetBeans

PERCHÉ PUÒ VALERE LA PENA USARLI Descrivere un diagramma come testo (invece che trascinando forme) rende i diagrammi **versionabili con Git** come il resto del codice, facili da aggiornare e da confrontare tra versioni diverse (diff testuale) — un vantaggio pratico non da poco quando i diagrammi UML sono usati in modalità "schizzo" e vanno aggiornati spesso, come visto nella Sezione 2.

4.3 Strumenti di modellazione UML

Strumenti per la progettazione UML e la generazione automatica di codice dai diagrammi UML:

- **Modelio** (open source) — modelio.org
- **ArgoUML** (open source) — argouml.org
- **Umbrello** (open source) — apps.kde.org/it/umbrello

EASYUML Plugin di NetBeans che genera codice dai diagrammi di classe e viceversa (i diagrammi di classe saranno approfonditi in una prossima lezione).

Riferimento del corso: M. Fowler, *UML Distilled*, 4ª ed., Pearson — Capitoli 1 e 9.

5 · Riepilogo per l'esame

Mappa concettuale in breve

Caso d'uso → testo strutturato (nome, attori, pre/post-condizioni, flusso normale, flussi alternativi) + modello grafico. Gli **attori** non sono necessariamente persone. I casi d'uso servono a elicitarre requisiti, comunicare con i clienti, scegliere i casi di test.

UML → standard OMG, linguaggio grafico di modellazione, indipendente dal linguaggio di programmazione, non è un processo. Si usa in forward o reverse engineering, e in tre modalità (sketch, blueprint, linguaggio di programmazione) — nella pratica quasi sempre come sketch, perché disegnare diagrammi completi richiede più tempo che scrivere codice.

Diagramma dei casi d'uso → vista d'insieme (attore = omino, caso d'uso = ellisse, associazione = linea semplice), **non sostituisce** le descrizioni testuali. Tre relazioni tra casi d'uso: «**extend**» (eccezione facoltativa, freccia tratteggiata verso il caso esteso), «**include**» (passo obbligatorio, freccia tratteggiata verso il caso incluso), **specializzazione** (caso particolare che sostituisce il generale, freccia solida a triangolo vuoto verso l'elemento generale). Un errore comune è confondere tipo/direzione delle linee. Per Fowler, il vero valore sta nel testo, non nel diagramma.

Strumenti → generali con forme UML (Visio, Omnigraffle, LibreOffice Draw, draw.io); diagram-as-code (PlantUML, Mermaid); modellazione con generazione di codice (Modelio, ArgoUML, Umbrello, EasyUML per NetBeans).

Domande di autoverifica

1. Quali sono i sei elementi della descrizione di un caso d'uso?

→ Nome univoco, attori partecipanti, pre-condizioni, post-condizioni, flusso di eventi normale, eventuali flussi di eventi alternativi.

2. Un attore in un caso d'uso deve essere per forza una persona?

→ No: se un sistema usa servizi offerti da altri sistemi, questi ultimi sono considerati a tutti gli effetti attori.

3. A cosa servono i casi d'uso, oltre che a descrivere i requisiti?

→ A comunicare con i clienti (grazie alla semplicità della notazione) e a scegliere i casi di test (la caratterizzazione del caso d'uso guida la definizione dei test).

4. Cosa significa l'acronimo UML, punto per punto?

→ Unified (standard riconosciuto a livello globale), Modeling (descrive un sistema ad alto livello di astrazione), Language (esprime concetti e idee in modo preciso e non ambiguo).

5. Che differenza c'è tra forward engineering e reverse engineering nell'uso di UML?

→ Nel forward engineering si disegnano i diagrammi UML prima di scrivere il codice (es. in fase di progettazione); nel reverse engineering si disegnano diagrammi UML a partire da codice esistente, per aiutarne la comprensione.

6. Perché UML viene usato quasi sempre in modalità "sketch" e raramente come linguaggio di programmazione?

→ Perché disegnare diagrammi con il livello di dettaglio richiesto per generare codice eseguibile richiede quasi sempre più tempo che scrivere direttamente il codice in un linguaggio di programmazione convenzionale.

7. Perché un diagramma dei casi d'uso da solo non basta?

→ Perché non sostituisce le descrizioni testuali dei casi d'uso: è utile solo se accompagnato dalle descrizioni vere e proprie, che contengono le informazioni più importanti (pre/post-condizioni, flussi).

8. In che verso punta la freccia in una relazione «extend», e cosa significa?

→ Punta verso il caso d'uso esteso (il caso base). Significa che, solo al verificarsi di determinate condizioni, l'esecuzione del caso base comporta anche l'esecuzione del caso che estende (es. AnnullamentoOperazione estende OperazioneBancomat).

9. In che verso punta la freccia in una relazione «include», e cosa significa?

→ Punta verso il caso d'uso incluso (utilizzato). Significa che l'esecuzione del caso principale comporta sempre anche l'esecuzione del caso incluso (es. OperazioneBancomat include InserimentoPIN).

10. In che verso punta la freccia in una relazione di specializzazione, e con che simbolo grafico?

→ Punta verso l'elemento più generale, con una linea solida terminante in un triangolo vuoto (non un semplice arrowhead pieno come extend/include).

11. Come si distingue graficamente una relazione «extend» da una «include»?

→ Sono identiche nella forma della linea (entrambe tratteggiate con freccia aperta): l'unica differenza visiva è l'etichetta testuale («extend» oppure «include»), quindi bisogna sempre controllarla con attenzione.

12. Nell'esempio Hospital Reception, perché Patient Hospital Admission include Patient Registration invece di estenderla?

→ Perché in quella fase (l'ammissione ospedaliera) la registrazione del paziente è obbligatoria: l'esecuzione di Patient Hospital Admission comporta sempre l'esecuzione di Patient Registration, che è la condizione che caratterizza «include» rispetto a «extend».

13. Cosa criticava Martin Fowler riguardo ai diagrammi dei casi d'uso?

→ Sosteneva che il diagramma dei casi d'uso è di scarsa utilità e che le energie vanno concentrate sul testo del caso d'uso, dove risiedono le informazioni più preziose; un rischio comune è complicare eccessivamente i casi d'uso, bloccando il processo di lavoro.

14. Che vantaggio pratico offrono gli strumenti "diagram as code" come PlantUML o Mermaid rispetto a un editor grafico?

→ Permettono di descrivere il diagramma come testo strutturato, rendendolo versionabile (es. con Git) insieme al codice, e più facile da aggiornare e confrontare tra versioni diverse rispetto a un disegno fatto trascinando forme.